Data Types and Addressing

Review

- **Memory layout**
  - Text, data (static and heap), and the stack
- **Procedure conventions**
- **Procedure call bookkeeping**
  - Caller Saved Registers:
    - Return address $ra
    - Arguments $a0, $a1, $a2, $a3
    - Return value $v0, $v1
    - $t Registers $t0 - $t9
  - Callee Saved Registers:
    - $s Registers $s0 - $s7
- **Procedure structure**
  - Prologue: allocate frame, save registers, assign locals
  - Body: procedure code
  - Epilogue: restore registers, free frame

Overview

- Data types
  - Application / HLL requirements
  - Hardware support (data and instructions)
- MIPS data types
- Support for bytes and strings
- Addressing Modes
  - Data
  - Instructions
- Large constants
- Far target addresses

Data Types

- Applications / HLL
  - Integer
  - Floating point
  - Character
  - String
  - Text
  - double precision
  - Signed, unsigned
  - (Pointers)
- Hardware support
  - Numeric data types
    - Integers
      - 8 / 16 / 32 / 64 bits
      - Signed or unsigned
      - Binary coded decimal
    - Floating point
      - 32 / 64 /128 bits
  - Nonnumeric data types
    - Characters
    - Strings
    - Boolean (bit maps)
    - Pointers

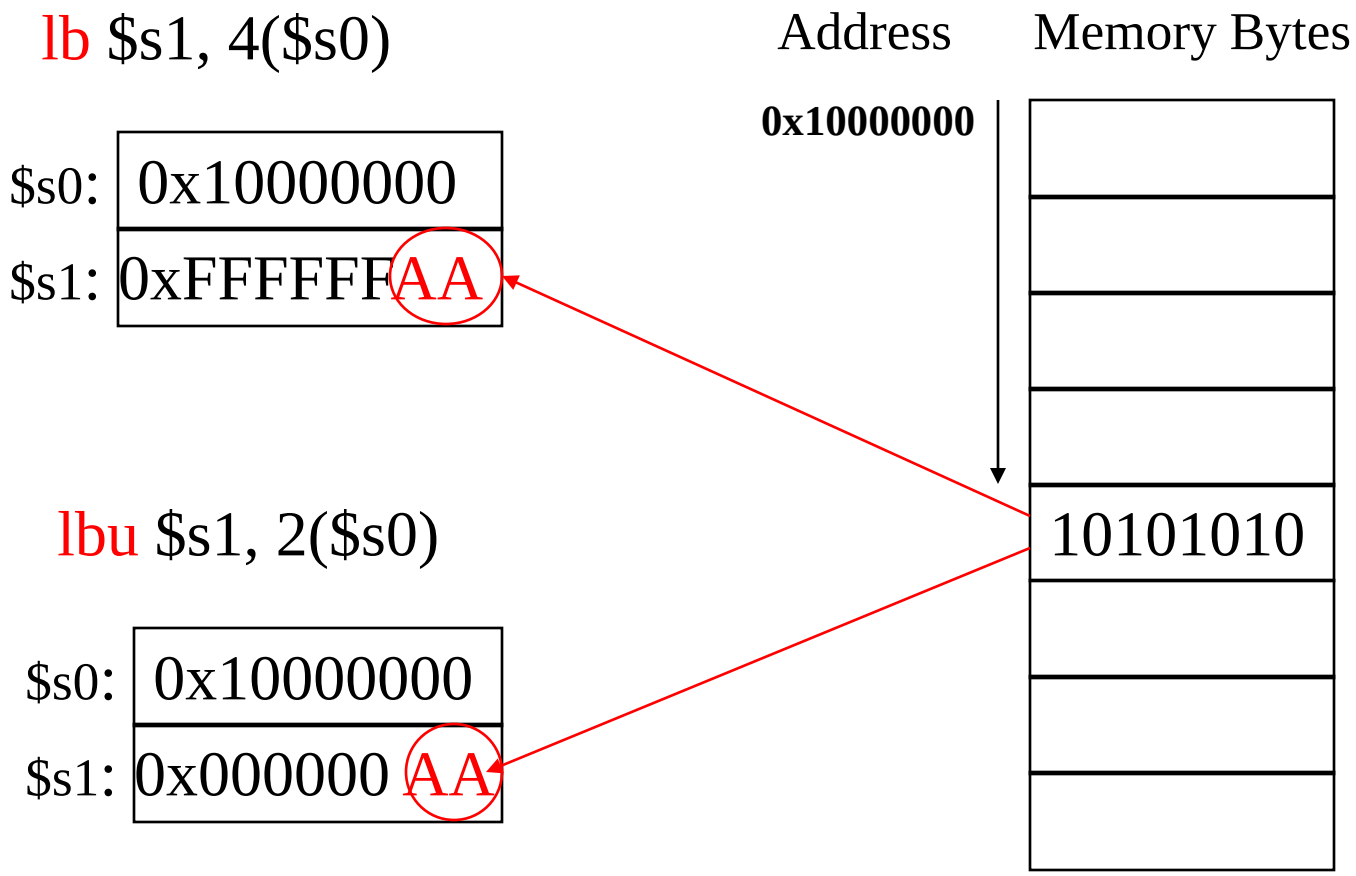
MIPS Data Types (1/2)

- Basic machine data type: 32-bit word (4 bytes)
  - 0100 0011 0100 1001 0101 0011 0100 0101
  - Integers (signed or unsigned)
    - 1,128,878,917 (How many bits? ± ??)
  - Floating point numbers
    - 201.32421875 (How many digits t left/right of floating point ?)
  - ASCII characters
    - C I S E
  - Memory addresses (pointers)
    - 0x43495345
  - Instructions

MIPS Data Types (2/2)

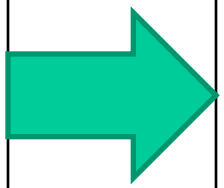
- 16-bit constants (immediates)
  - addi $s0, $s1, 0x8020
  - lw $t0, 20($s0)
- Half word (16 bits)
  - **lh** (**lhu**): load half word lh $t0, 20($s0)
  - **sh**: save half word sh $t0, 20($s0)
- Byte (8 bits)
  - **lb** (**lbu**): load byte lb $t0, 20($s0)
  - **sb**: save byte sb $t0, 20($s0)

Byte Instructions



String Manipulation

```
Void strcpy (char[], char y[]) {
  int i;
  i = 0;
  while ((x[i]=y[i]) != 0)
     i = i + 1;
}
```



```
strcpy:
      subi $sp, $sp, 4
      sw   $s0, 0($sp)
      add  $s0, $zero, $zero
L1:   add $t1, $a1, $s0
      lb    $t2, 0($t1)
      add  $t3, $a0, $s0
      sb    $t2, 0($t3)
      beq  $t2, $zero, L2
      addi $s0, $s0, 1
      j       L1
L2:  lw   $s0, 0($sp)
      addi $sp, $sp, 4
      jr      $ra
```

C convention:

Null byte (00000000)
represents end of the string

Constants

- Small constants are used frequently (50% of operands)
  - e.g., A = A + 5;
- Solutions
  - Put 'typical constants' in memory and load them.
  - Create hard-wired registers (like $zero) for constants.
- MIPS Instructions:

```
slti $8, $18, 10
andi $29, $29, 6
ori  $29, $29, 0x4a
addi $29, $29, 4
```

8	29	29	4

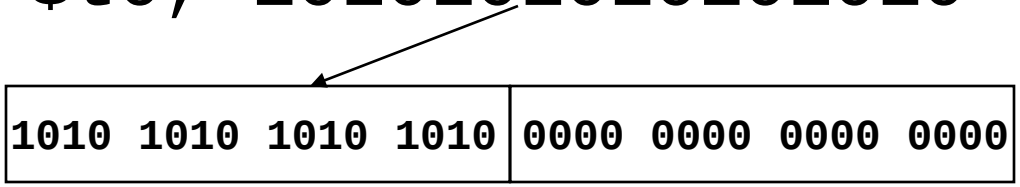
101011	10011	01000	0000 0000 000 0100

Large Constants

- To load a 32 bit constant into a register:

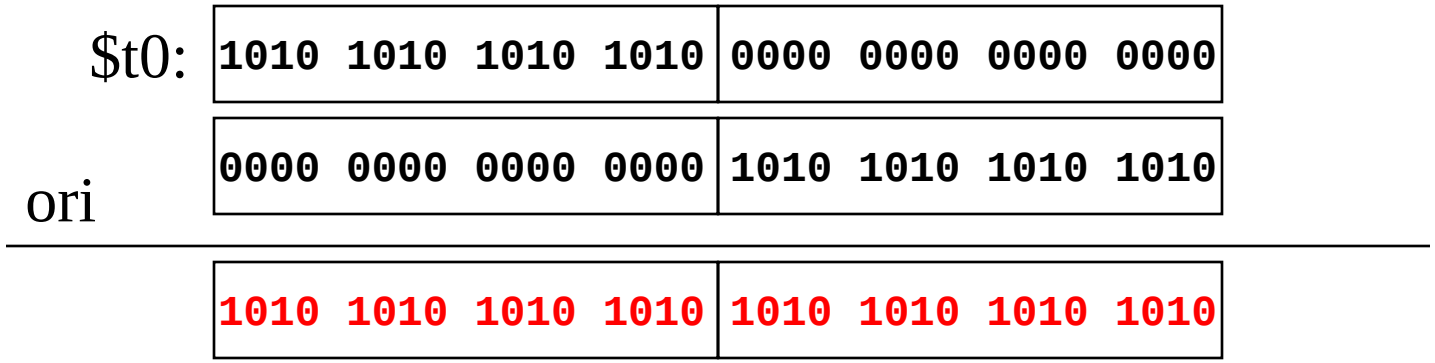
1. Load (16) higher order bits

```
lui $t0, 1010101010101010
```



2. Then must get the lower order bits right, i.e.,

```
ori $t0, $t0, 1010101010101010
```



Review/Overview

- **Data types and addressing** included in the ISA
  - Compromise between application requirements and hardware implementation
- **MIPS data types**
  - 32-bit word
  - 16-bit half word
  - 8-bit bytes
- **Addressing Modes**
  - Data
    - Register
    - 16-bit signed constants (immed)
    - Base addressing
  - Instructions
    - PC-relative
    - (Pseudo) direct

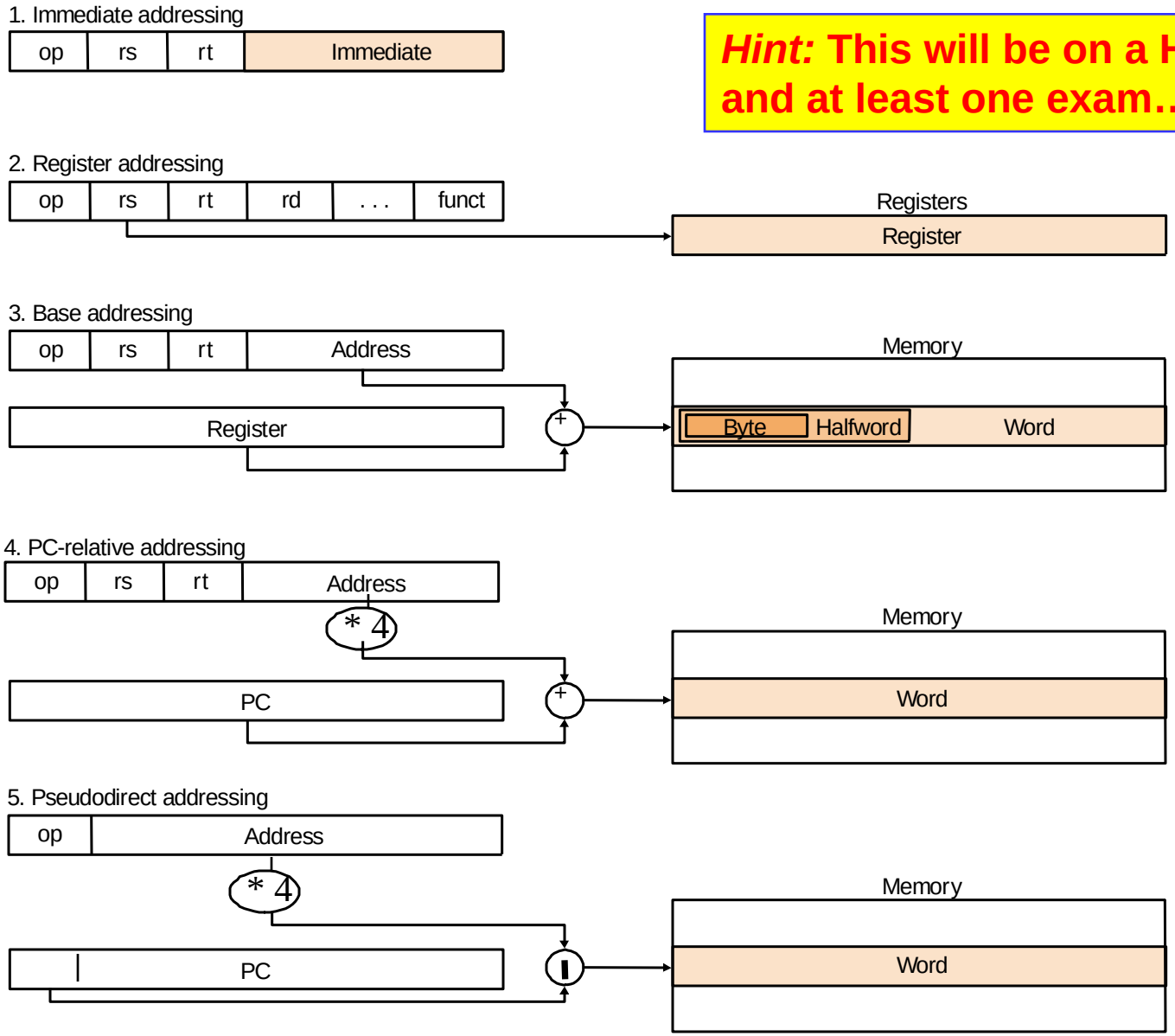
Data Addressing Modes

- Addresses for *data* and *instructions*
- Data (operands and results)
  - Registers
  - Base addressed Memory locations (pointers)
  - Constants (immed)
- Efficient encoding of addresses (space: 32 bits)
  - Registers (32) => 5 bits to encode address
  - *Destructive* instructions: reg2 = reg2 + reg1
  - Accumulator

Data Addressing Modes

- **Register addressing**
  - The most common (fastest and shortest)
  - add $3, $2, $1
- **Base addressing**
  - Operand is at a memory location with offset
  - lw $t0, 20 ($t1)
- **Immediate addressing**
  - Operand is a small constant within the instruction
  - addi $t0, $t1, 4 (signed 16-bit integer)

MIPS Addressing Modes

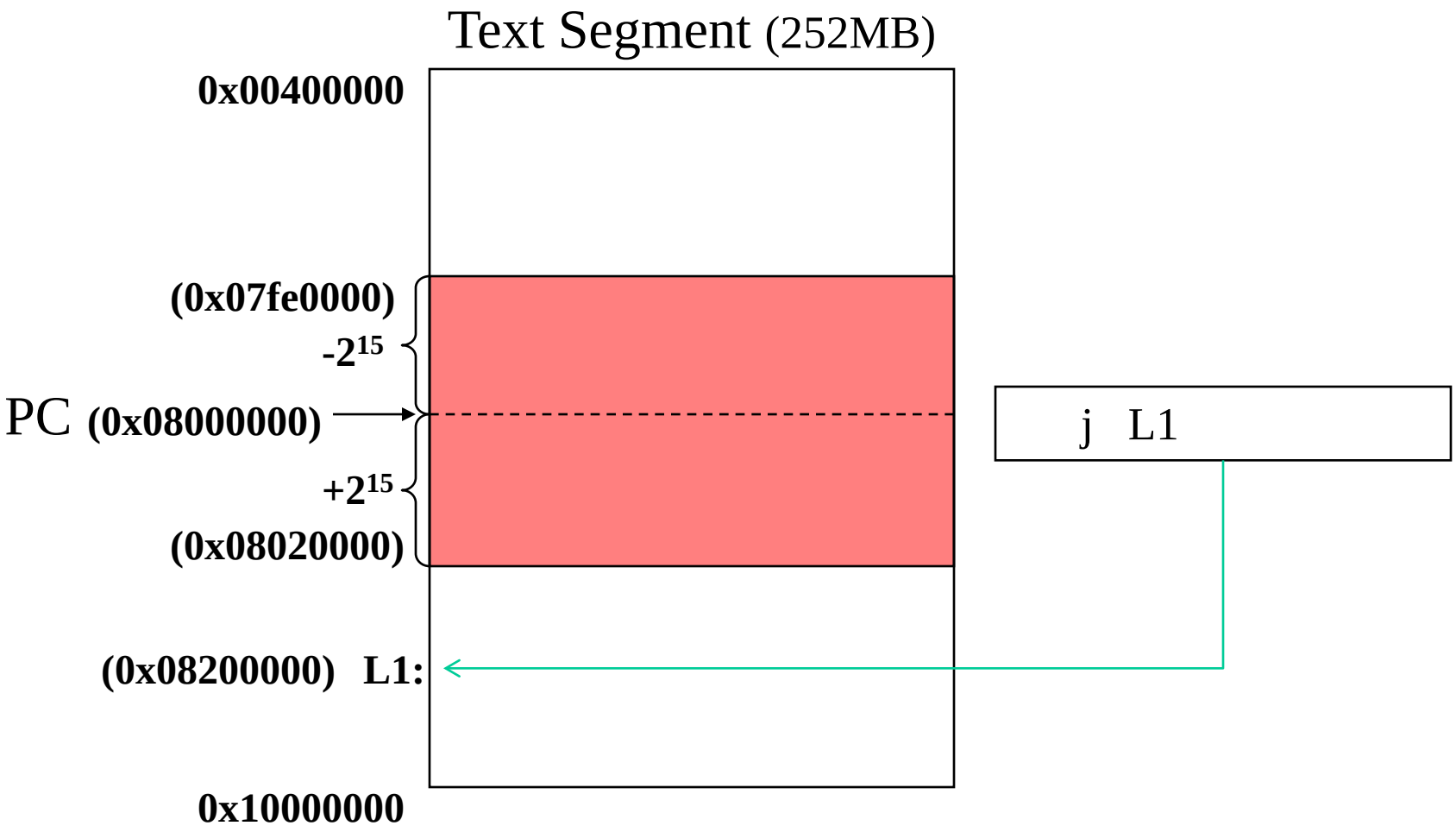


***Hint:* This will be on a Homework and at least one exam…**

Instruction Addressing Modes

- Addresses are 32 bits long
- Special purpose register PC (**program counter**) stores the address of the current instruction
- **PC-relative addressing** (branches)
  - Address: PC + (constant in the instruction) * 4
  - beq $t0, $t1, 20 (0x15090005)
- **Pseudodirect addressing** (jumps)
  - Address: PC[31:28] **:** (constant in the instruction) * 4

Far Target Address



Producing Machine Code

- **Simple case**
  - Arithmetic, logical, shifts, etc.
  - All necessary info is within the instruction already.
- **Conditional branches** (beq, bne)
  - Once pseudoinstructions are replaced by real ones, we know by how many instructions for branch span
  - PC-relative, easy to handle
- **Direct (absolute) addresses**
  - Jumps (j and jal)
  - Direct (absolute) references to data

Overview

- Pointers (addresses) and values
- Argument passing
- Memory lifetime and scope
- Pointers and arrays
- Pointers in MIPS

Pointers

- **Pointer:** a variable that contains the address of another variable
  - HLL version of machine language memory address
- Why use Pointers?
  - Sometimes only way to express computation
  - Often more compact and efficient code
- Why not?
  - Huge source of bugs in real software, perhaps the largest single source

  1) Dangling reference (premature free)
  2) Memory leaks (tardy free): can't have long-running jobs without periodic restart of them

C Pointer Operators

- Suppose `c` has value 100, it is located in memory at address 0x10000000
- Unary operator & gives address:
  `p = &c;` gives address of `c` to `p`;
  - `p` "points to" `c` `(p == 0x10000000)` (Referencing)
- Unary operator `*` gives value that pointer points to
  - if `p = &c` => `*p == 100` (Dereferencing a pointer)
- Deferencing ⇒ data transfer in assembler
  - `... = ... *p ...;` ⇒ load
    (get value from location pointed to by p)
  - `*p = ...;` ⇒ store
    (put value into location pointed to by p)

Assembly Code

`c` is `int`, has value 100, in memory at address 0x10000000, `p` in `$a0`, `x` in `$s0`

```
1.  p = &c; /* p gets 0x10000000*/
    lui $a0,0x1000 # p = 0x10000000
2.  x = *p;  /* x gets 100 */
    lw  $s0, 0($a0) # dereferencing p
3. *p = 200; /* c gets 200 */
    addi $t0,$0,200
    sw   $t0, 0($a0) # dereferencing p
```

Example

```
int strlen(char *s) {
  char *p = s;          /* p points to chars */
  while (*p != '\0')
    p++;                /* points to next char */
  return p - s;         /* end - start */
}
```

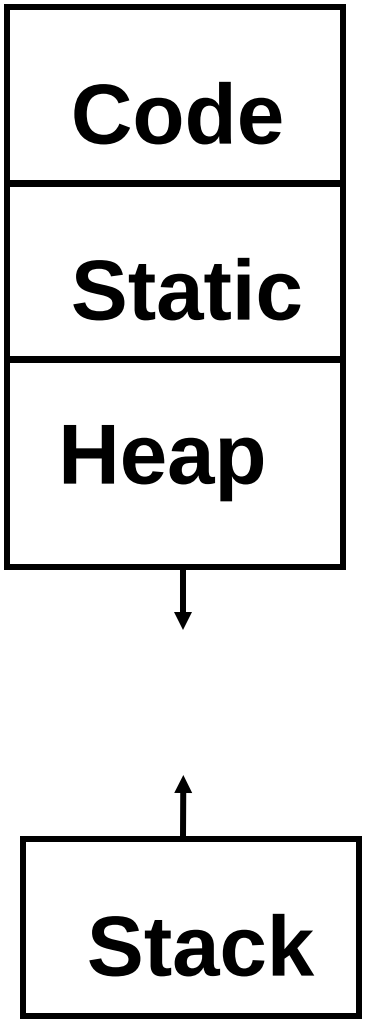
```
     mov  $t0,$a0
     lbu  $t1,0($t0) /* derefence p */
     beq  $t1,$zero, Exit
Loop:addi $t0,$t0,1  /* p++ */
     lbu  $t1,0($t0) /* derefence p */
     bne  $t1,$zero, Loop
Exit:sub  $v0,$t0,$a0
     jr   $ra
```

Argument Passing Options

- 2 choices
  - "Call by Value": pass a copy of the item to the function/procedure
  - "Call by Reference": pass a pointer to the item to the function/procedure
- Single word variables passed by value
- Passing an array? e.g., `a[100]`
  - Pascal (call by value) copies 100 words of `a[]` onto the stack
  - C (call by reference) passes a pointer (1 word) to the array `a[]` in a register

Review: Lifetime of Storage and Scope

- Automatic (stack allocated)
  - Typical local variables of a function
  - Created upon call, released upon return
  - Scope is the function
- Heap allocated
  - Created upon malloc, released upon free
  - Referenced via pointers
- External / static
  - Exist for entire program



Arrays, Pointers, and Functions

- 4 versions of array function that adds two arrays and puts sum in a third array (*sumarray*)
  1. Third array is passed to function as argument
  2. Using a local array (on stack) for result and passing a pointer to it
  3. Third array is allocated on heap
  4. Third array is declared static
- Purpose of example is to show interaction of C statements, pointers, and memory allocation

Version 1

```
int x[100], y[100], z[100];
sumarray(x, y, z);
```

- C calling convention means:

```
sumarray(&x[0], &y[0], &z[0]);
```

- Really passing pointers to arrays

```
addi $a0,$gp,0   # x[0] starts at $gp
addi $a1,$gp,400 # y[0] above x[100]
addi $a2,$gp,800 # z[0] above y[100]
jal  sumarray
```

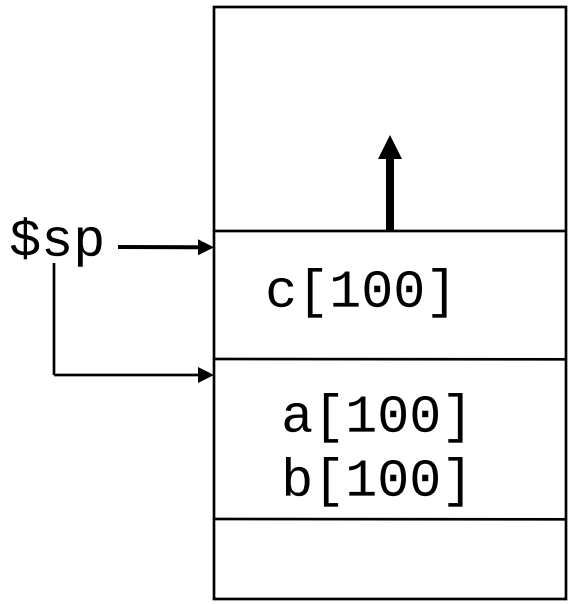
Version 1: Compiled Code

```
void sumarray(int a[], int b[], int c[]) {
    int i;
    for(i = 0; i < 100; i = i + 1)
            c[i] = a[i] + b[i];
}
```

```
            addi    $t0,$a0,400     # beyond end of a[]
Loop:       beq     $a0,$t0,Exit
            lw      $t1, 0($a0)     # $t1=a[i]
            lw      $t2, 0($a1)     # $t2=b[i]
            add     $t1,$t1,$t2     # $t1=a[i] + b[i]
            sw      $t1, 0($a2)     # c[i]=a[i] + b[i]
            addi    $a0,$a0,4       # $a0++
            addi    $a1,$a1,4       # $a1++
            addi    $a2,$a2,4       # $a2++
            j       Loop
Exit:       jr      $ra
```

Version 2

```
int *sumarray(int a[],int b[]) {
    int i, c[100];
    for(i=0;i<100;i=i+1)
        c[i] = a[i] + b[i];
    return c;
}
```

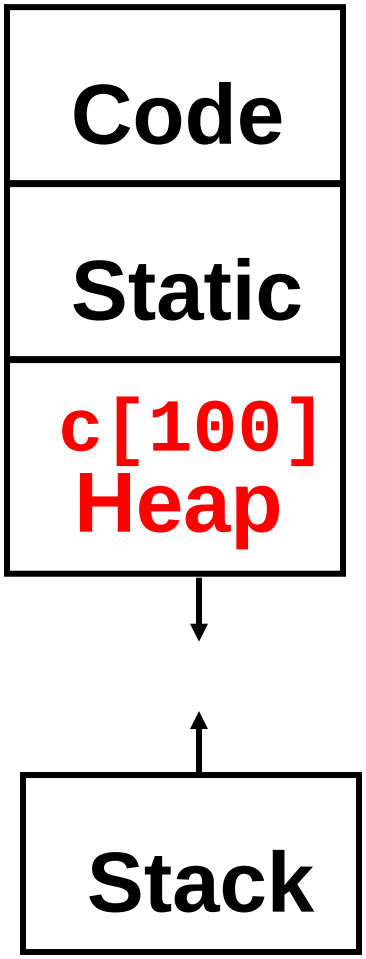



```
        addi  $t0,$a0,400   # beyond end of a[]
        addi  $sp,$sp,-400  # space for c
        addi  $t3,$sp,0     # ptr for c
        addi  $v0,$t3,0     # $v0 = &c[0]
Loop:   beq   $a0,$t0,Exit
        lw    $t1, 0($a0)   # $t1=a[i]
        lw    $t2, 0($a1)   # $t2=b[i]
        add   $t1,$t1,$t2   # $t1=a[i] + b[i]
        sw    $t1, 0($t3)   # c[i]=a[i] + b[i]
        addi  $a0,$a0,4     # $a0++
        addi  $a1,$a1,4     # $a1++
        addi  $t3,$t3,4     # $t3++
        j     Loop
Exit:   addi  $sp,$sp, 400  # pop stack
        jr    $ra
```

Version 3

```
int *sumarray(int a[],int b[]) {
        int i;
        int *c;
        c = (int *) malloc(100);
        for(i=0;i<100;i=i+1)
           c[i] = a[i] + b[i];
        return c;
}
```

- **Heap space not reused unless freed**
  - Can lead to memory leaks
  - Java, Scheme have garbage collectors to reclaim free space



Version 3: Compiled Code

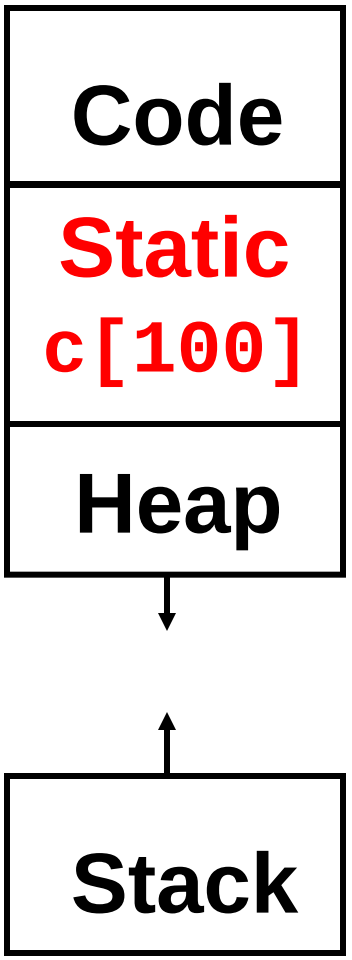
```
    addi  $t0,$a0,400 # beyond end of a[]
    addi  $sp,$sp,-12 # space for regs
    sw    $ra, 0($sp) # save $ra
    sw    $a0, 4($sp) # save 1st arg.
    sw    $a1, 8($sp) # save 2nd arg.
    addi  $a0,$zero,400
    jal   malloc
    addi  $t3,$v0,0   # ptr for c
    lw    $a0, 4($sp) # restore 1st arg.
    lw    $a1, 8($sp) # restore 2nd arg.
Loop:   beq  $a0,$t0,Exit
    ... (loop as before on prior slide )
    j     Loop
Exit:lw   $ra, 0($sp)  # restore $ra
    addi  $sp, $sp, 12 # pop stack
    jr    $ra
```

Version 4

```
int *sumarray(int a[],int b[]) {
     int i;
     static int c[100];

     for(i=0;i<100;i=i+1)
             c[i] = a[i] + b[i];
     return c;
}
```

- Compiler allocates once for function,
  - Will be changed next time sumarray invoked



Review: MIPS Instructions

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1, $s2, $s3`	`$s1 = $s2 + $s3`	Three operands; data in registers
	subtract	`sub $s1, $s2, $s3`	`$s1 = $s2 - $s3`	Three operands; data in registers
	add immediate	`addi $s1, $s2, 100`	`$s1 = $s2 + 100`	Used to add constants
Data transfer	load word	`lw $s1, 100($s2)`	`$s1 = Memory[$s2 + 100]`	Word from memory to register
	store word	`sw $s1, 100($s2)`	`Memory[$s2 + 100] = $s1`	Word from register to memory
	load byte	`lb $s1, 100($s2)`	`$s1 = Memory[$s2 + 100]`	Byte from memory to register
	store byte	`sb $s1, 100($s2)`	`Memory[$s2 + 100] = $s1`	Byte from register to memory
	load upper immediate	`lui $s1, 100`	$s1 = 100 * 2^{16}	Loads constant in upper 16 bits
Conditional branch	branch on equal	`beq $s1, $s2, 25`	if (`$s1 == $s2`) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	`bne $s1, $s2, 25`	if (`$s1 != $s2`) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	`slt $s1, $s2, $s3`	if (`$s2 < $s3`) `$s1` = 1; else `$s1` = 0	Compare less than; for beq, bne
	set less than immediate	`slti $s1, $s2, 100`	if (`$s2 < 100`) `$s1` = 1; else `$s1` = 0	Compare less than constant
Uncondi-tional jump	jump	`j 2500`	go to 10000	Jump to target address
	jump register	`jr $ra`	go to `$ra`	For switch, procedure return
	jump and link	`jal 2500`	`$ra` = PC + 4; go to 10000	For procedure call

Conclusions

- ***Data can be anything***
  - Datatyping restricts data representations
  - Applications restrict datatyping
- **MIPS Datatypes:** Number, String, Boolean
- **Addressing:** Pointers, Values
  - Many addressing modes (direct, indirect,…)
  - Memory-based address storage (`jr` instruction)
- **Arrays:** *big chunks of memory*
  - Pointers versus stack storage
  - Be careful of memory leaks!